Design and Analysis of Algorithms Project Report

Anshul Raj Singh, Gauransh Kapur

October 21, 2026

Contents

1 Problem statement and test cases **1**

2 Solution **2**
2.1 Recurrence for $G(v)$ 2
2.2 Recurrence for $H(v, d)$ 2
2.3 Pseudocode 3

3 Analysis of Algorithm **3**
3.1 Time Complexity 3
3.2 Space Complexity 4

4 Python Code **5**

5 Conclusion **5**

1 Problem statement and test cases

One day, Alex got tired of climbing in a gym and decided to take a very large group of climbing friends outside to climb. The climbing area where they went, had a huge wide boulder, not very tall, with various marked hand and foot holds. Alex quickly determined an "allowed" set of moves that her group of friends can perform to get from one hold to another. The overall system of holds can be described by a rooted tree T with n vertices, where each vertex corresponds to a hold and each edge corresponds to an allowed move between holds. The climbing paths converge as they go up the boulder, leading to a unique hold at the summit, represented by the root of T. Alex and her friends (who are all excellent climbers) decided to play a game, where as many climbers as possible are simultaneously on the boulder and each climber needs to perform a sequence of exactly k moves. Each climber can choose an arbitrary hold to start from, and all moves must move away from the ground. Thus, each climber traces out a path of k edges in the tree T, all directed toward the root. However, no two climbers are allowed to touch the same hold; the paths followed by different climbers cannot intersect at all.

Describe and analyze an efficient dynamic programming algorithm to compute the maximum number of climbers that can play this game.

2 Solution

Let r be the root of the tree T. We define two quantities for each vertex v in T.

- $G(v)$ – maximum number of paths contained in the subtree rooted at v, with v unused.
- $H(v, d)$ – maximum number of paths in the subtree, subject to a distinguished path that starts d edges below v and travels upward through v. When $d = k$, the distinguished path ends at v and counts as one complete path.
- $H(v, 0) = G(v)$ – the distinguished path starts at v, but has not yet been completed within the subtree.
- An impossible state has value $-\infty$.

Then the required answer is

$$\max\{G(r), H(r, k)\}.$$

So we need to compute $G(v)$ and $H(v, d)$ for all vertices v in T and for all $1 \leq d \leq k$. We proceed via recursion.

2.1 Recurrence for $G(v)$

Let $u_1, \ldots, u_m$ be the children of v. For every $1 \leq j \leq m$, the maximum number of k-length paths that end at u_j is $\max\{G(u_j), H(u_j, k)\}$. Thus,

$$G(v) = \sum_{j=1}^{m} \max\{G(u_j), H(u_j, k)\}.$$

2.2 Recurrence for $H(v, d)$

For $1 \leq d < k$, let u be that child of v such that the distinguished path goes through u. Then, the maximum number of k-length paths that can be chosen in the subtree, keeping u fixed, is

$$H(u, d-1) + \sum_{\substack{w \text{ child of } v \\ w \neq u}} \max\{G(w), H(w, k)\}.$$

Therefore, we have the following recurrence

$$H(v, d) = \max_{u \text{ child of } v} \left(H(u, d-1) + \sum_{\substack{w \text{ child of } v \\ w \neq u}} \max\{G(w), H(w, k)\} \right)$$

for $1 \leq d < k$.

When $d = k$, then the distingueshed path finishes at v, so this path must be counted in the final sum. This gives

$$H(v, k) = 1 + \max_{u \text{ child of } v} \left(H(u, d-1) + \sum_{\substack{w \text{ child of } v \\ w \neq u}} \max\{G(w), H(w, k)\} \right).$$

So, our final answer is

$$\max\{G(r), H(r, k)\}.$$

2.3 Pseudocode

Below is the pseudocode for the algorithm.

Algorithm 1 Maximum Climbers Calculation

ALGORITHM MaximumClimbers(T, r, k)
Input: Rooted tree T with root r, Positive integer k
Output: Maximum number of vertex-disjoint upward paths of exactly k edges

```
for each vertex v, in postorder do
    G[v] ← 0
    for each child u of v do
        G[v] ← G[v] + max(G[u], H[u, k])
    end for
    H[v, 0] ← G[v]
    for d ← 1 to k do
        H[v, d] ← −∞
        for each child u of v do
            other ← G[v] − max(G[u], H[u, k])
            if H[u, d − 1] ≠ −∞ then
                candidate ← H[u, d − 1] + other
                if d = k then
                    candidate ← candidate + 1
                end if
                H[v, d] ← max(H[v, d], candidate)
            end if
        end for
    end for
end for
```

return $\max(G[r], H[r, k])$

3 Analysis of Algorithm

3.1 Time Complexity

Let n be the number of vertices in the rooted tree T, and let k be the required number of edges in each climbing path.

- *Processing the vertices* – The outer loop processes every vertex exactly once, in postorder. Hence, it executes n times.
- *Computing* $G(v)$ – For each vertex v, the algorithm iterates over all children u of v to compute $G(v)$. Each iteration requires constant time, since it evaluates $\max\{G(u), H(u, k)\}$ and performs an addition.

Let $c(v)$ denote the number of children of vertex v. The total time spent computing all values $G(v)$ is therefore

$$\sum_{v \in T} O(c(v)).$$

In a rooted tree with n vertices, every vertex except the root has exactly one parent. Consequently, the total number of child relationships is $n-1$, so

$$\sum_{v \in T} c(v) = n - 1.$$

It follows that computing all values $G(v)$ takes

$$O\left(\sum_{v \in T} c(v)\right) = O(n).$$

- *Computing $H(v, d)$* – For each vertex v, the algorithm computes the states $H(v, d)$ for $d = 1, \ldots, k$. Thus, there are k iterations of the outer depth loop for each vertex.

For each pair (v, d), the algorithm iterates over every child u of v. Each iteration takes constant time since the value 'other' is computed using the already computed value $G(v)$, and the remaining operations involve comparisons, additions, and assignments. The algorithm does not iterate over the other children again to calculate their contribution.

Therefore, the total time spent computing all values $H(v, d)$ is

$$\begin{aligned}\sum_{v \in T} \sum_{d=1}^{k} O(c(v)) &= O\left(k \sum_{v \in T} c(v)\right) \\ &= O(k(n-1)) \\ &= O(nk).\end{aligned}$$

- *Computing the final answer* – After all states have been computed, the algorithm returns $\max\{G(r), H(r, k)\}$, which requires $O(1)$ time.

Combining all the costs, the total running time is

$$\begin{aligned}T(n, k) &= O(n) + O(nk) + O(1) \\ &= O(nk).\end{aligned}$$

Thus, the time complexity of the algorithm is

$$\boxed{O(nk)}.$$

3.2 Space Complexity

We now analyze the additional memory required by Algorithm.

- *Dynamic programming tables* – The algorithm stores one value $G(v)$ for each vertex v, requiring $O(n)$ space. It also stores $H(v, d)$ for every vertex v and every $d \in \{0, 1, \ldots, k\}$. There are $n(k+1)$ such states, so the space required for this table is

  $$O(n(k+1)) = O(nk)$$

  for $k \geq 1$.

- *Tree representation* – The rooted tree can be represented using a list of children for each vertex. Since a tree with n vertices has $n-1$ edges, this representation requires $O(n)$ space.
- *Auxiliary variables* – The variables used inside the loops, such as 'other' and 'candidate', require $O(1)$ additional space at any one time.

The algorithm therefore uses

$$\begin{aligned} S(n,k) &= O(n) + O(nk) + O(1) \\ &= O(nk). \end{aligned}$$

Hence, the space complexity of the dynamic programming tables and tree representation is

$$\boxed{O(nk)}.$$

4 Python Code

The complete Python code for the algorithm is available in the project repository:

`https://github.com/anshulmath/DAA_Project`

5 Conclusion